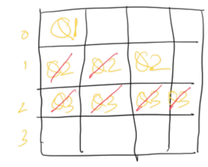
N-Queens

n x n board

n = 4

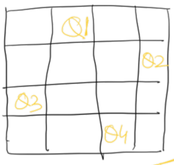
we need to place n queens so that they don't attack each other



we can't place only Q3 so we will backtrack

again no safe space for Q4 so we'll backtrack

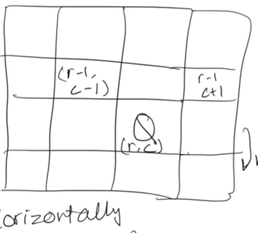
we can't place Q3 also ahead so we'll backtrack further and we notice that Q2 can't be moved any further so we'll backtrack to Q1 and move that



1. we want to place n queens in n rows
2. we will only place the queen where she is safe

Code

```
void nQueens(board[][], row, n, ans){
    if (row==n) ans.pb(board)
                return
    for(j=0 to n){
      if(isSafe(board, row, j, n){
          board[row][j] = 'Q'
          nQueens(board, row+1, n)
          board[row][j] = '.'  // backtracking step, now when it backtracks the place where it was at should be marked as empty and position should be incremented
      }
    }
```



we will not consider below diagonals since they don't exist

1. Horizontally

```
for(j=0 to n)
    if (B[row][j]==='Q')
        return false
```

} O(n)

2. Vertically

```
for(i=0 to n)
    if (B[i][col]==='Q')
        return false
```

3. Left diag

```
for(i=r, j=c; i>=0 && j>=0; i--,j--)
    if (B[i][j]==='Q')
        return false
```

4. Right diag

```
for(i=r, j=c; i>=0 && j<n; i--,j++)
    if (B[i][j]==='Q')
        return false
```

To initialise empty board

```
vector<string> board(n, string(n,'.'))
```

Total TC

n Queens to be placed in n diff rows

O(n) choices

O(n-1) choices

O(n-2) choice

O(n-3) choices

...

$\therefore O(n!)$